

OpenCode 后端架构学习笔记

分支： `learn/backend-core`

核心包： `packages/opencode/src/`

目录

1. 整体架构一览
2. 启动流程
3. Agent 系统
4. 主循环 (loop)
5. LLM 调用层
6. 流式处理器 (Processor)
7. 工具系统 (Tools)
8. 权限系统
9. 消息与会话模型
10. 上下文压缩 (Compaction)
11. 插件系统 (Plugin)
12. 关键数据流图
13. 文件速查表

1. 整体架构一览



三个关键层:

层	文件	职责
编排层	session/prompt.ts	循环调度, 决定什么时候调用 LLM、什么时候执行子任务、什么时候压缩
流处理层	session/processor.ts	消费 LLM 的流式事件, 实时把每个片段写入数据库
传输层	session/llm.ts	组装参数, 调用 AI SDK, 屏蔽 Provider 差异

2. 启动流程



运行方式：

```
bun dev .           # TUI 模式（终端交互界面）
bun run serve       # 纯 HTTP API 服务器
```

3. Agent 系统

文件： `packages/opencode/src/agent/agent.ts`

Agent 就是 AI 的"角色"。不同 Agent 有不同的权限、提示词、模型。

内置 Agent 一览

Agent	模式	用途	关键权限
build	primary	默认主 Agent	允许所有工具，包括 question、plan_enter
plan	primary	计划模式	禁止编辑文件（除 plans/*.md）

Agent	模式	用途	关键权限
general	subagent	通用子 Agent，被 TaskTool 调度	禁用 todo 工具
explore	subagent	只读探索代码库	只允许 grep/glob/read/bash
compaction	primary (隐藏)	压缩上下文	禁止所有工具
title	primary (隐藏)	生成会话标题	禁止所有工具
summary	primary (隐藏)	生成消息摘要	禁止所有工具

Agent 权限合并规则

```
defaults (系统默认)
+ agent 内置配置
+ 用户 opencode.json 中的 agent 配置
↓
PermissionNext.merge() 深度合并，后者覆盖前者
```

Agent 模式

- primary：可以直接被用户使用
- subagent：只能被 TaskTool 派发调用
- all：两者均可（用户自定义 Agent 默认）

自定义 Agent

在 .opencode/config.json 或全局配置中：

```
{
  "agent": {
    "mybot": {
      "prompt": "你是一个专门写测试的 Agent...",
      "model": "anthropic/claude-sonnet-4-5",
      "mode": "primary",
      "steps": 10,
      "permission": { "bash": "deny" }
    }
  }
}
```

4. 主循环 (loop)

文件： `packages/opencode/src/session/prompt.ts` → `loop()`

这是整个系统最核心的函数。每一次迭代对应一次 LLM 调用。

循环的 9 个步骤

```
while (true) {  
    步骤1: 读取消息历史 (filterCompacted 跳过已压缩的旧消息)  
    步骤2: 找到最近的 user / assistant / finished 消息  
    步骤3: 检查退出条件  
        → LLM 已给出最终回复 (finish ≠ tool-calls/unknown) → break  
        → 用户取消 (abort) → break  
    步骤4a: 处理待执行的 subtask (子 Agent 任务)  
        → 直接执行 TaskTool, 不经过 LLM → continue  
    步骤4b: 处理待执行的 compaction (上下文压缩)  
        → 执行压缩 → continue  
    步骤5: 检查上下文是否溢出  
        → 创建 compaction 任务 → continue  
    步骤6: 构建本轮参数 (工具、系统提示词、消息历史)  
    步骤7: 调用 SessionProcessor.process()  
    步骤8: 处理结构化输出 (json_schema 模式)  
    步骤9: 根据 process() 返回值决定  
        → "continue" : 继续下一轮  
        → "stop"      : 退出循环  
        → "compact"   : 创建压缩任务, 继续  
}
```

循环退出条件

```
// 退出条件: LLM 已完成 (不是在等待工具结果)  
if (  
    lastAssistant?.finish &&  
    !["tool-calls", "unknown"].includes(lastAssistant.finish) &&  
    lastUser.id < lastAssistant.id // assistant 消息比 user 消息更新  
) {  
    break  
}
```

多轮对话中的"排队"机制

当 LLM 还在运行时, 用户发了新消息, 新消息不会打断当前运行, 而是:

1. 被保存到数据库
2. 当前 loop 在下一轮会读到这条新消息
3. 用 <system-reminder> 包裹提醒 LLM 处理它

5. LLM 调用层

文件： `packages/opencode/src/session/llm.ts` → `LLM.stream()`

系统提示词组装顺序

```
agent.prompt (如果有自定义)
OR
SystemPrompt.provider(model) (Provider 默认提示词)
+
input.system (本轮额外注入，如结构化输出提示)
+
user.system (用户消息中附带的自定义 system)
```

合并后保持"2段结构"——第一段用于 Anthropic 的提示词缓存 (`cache_control`), 可以节省 token 费用。

模型参数合并优先级 (低→高)

```
ProviderTransform.options() (基础默认值)
  mergeDeep
model.options (模型级别配置)
  mergeDeep
agent.options (Agent 级别配置)
  mergeDeep
variant (模型变体, 如 extended-thinking)
```

工具调用自动修复

LLM 有时会用错误的工具名调用工具，系统有两级修复：

1. **大小写修复**： `Read` → `read` (自动转小写)
2. **Fallback**： 不认识的工具名 → 替换为 `invalid` 工具，向 LLM 报告错误

特殊 Provider 处理

Provider	特殊处理
OpenAI OAuth（Codex/GitHub Copilot）	system 通过 options.instructions 传，不用 system message
LiteLLM 代理	消息历史有工具调用时，必须传一个 dummy _noop 工具
GitHub Copilot	不限制 maxOutputTokens

6. 流式处理器（Processor）

文件： packages/opencode/src/session/processor.ts → SessionProcessor

事件类型与处理

LLM.stream() 返回的流，每个事件都有 type：

事件	含义	处理方式
start	流开始	设置 session 状态为 "busy"
reasoning-start/delta/end	思维链（CoT）	创建/更新 reasoning Part
tool-input-start	工具调用开始	创建 pending 状态的 ToolPart
tool-call	工具参数就绪	更新为 running，检测死循环
tool-result	工具执行完毕	更新为 completed，写入输出
tool-error	工具执行出错	更新为 error，检查是否权限拒绝
text-start/delta/end	文本输出	创建/增量更新 text Part
start-step	一个步骤开始	创建文件快照（用于 diff）
finish-step	一个步骤结束	计算 token 费用，生成 patch Part
error	流级别错误	抛出让外层处理

一条 Assistant 消息的结构

```
AssistantMessage
├─ Part: step-start      (步骤开始, 含快照 ID)
├─ Part: reasoning      (思维链文本)
├─ Part: text           (LLM 输出的文本)
├─ Part: tool (pending) → (running) → (completed/error)
├─ Part: step-finish    (本步骤 token 消耗)
└─ Part: patch          (本步骤修改了哪些文件)
```

死循环检测 (Doom Loop)

```
// 连续 3 次相同工具 + 相同参数 = 疑似死循环
const DOOM_LOOP_THRESHOLD = 3

if (lastThree.every(p =>
  p.tool === toolName &&
  JSON.stringify(p.input) === JSON.stringify(currentInput)
)) {
  // 向用户发出权限询问, 让用户决定是否继续
  await PermissionNext.ask({ permission: "doom_loop", ... })
}
```

返回值含义

返回值	含义
"continue"	正常完成, loop 继续下一轮
"stop"	权限拒绝或错误, loop 退出
"compact"	token 超限, loop 去执行压缩

7. 工具系统 (Tools)

文件: packages/opencode/src/tool/registry.ts

工具的接口

```
// 每个工具都是一个 Tool.Info 对象:
{
  id: string,
  init: async (ctx) => {
    parameters: ZodSchema,    // 参数 schema (验证 LLM 传入的参数)
    description: string,      // 告诉 LLM 这个工具的用途
    execute: async (args, ctx) => {
      output: string,         // 工具输出 (回传给 LLM)
      title: string,          // UI 显示标题
      metadata: any,          // 附加元数据
    }
  }
}
```

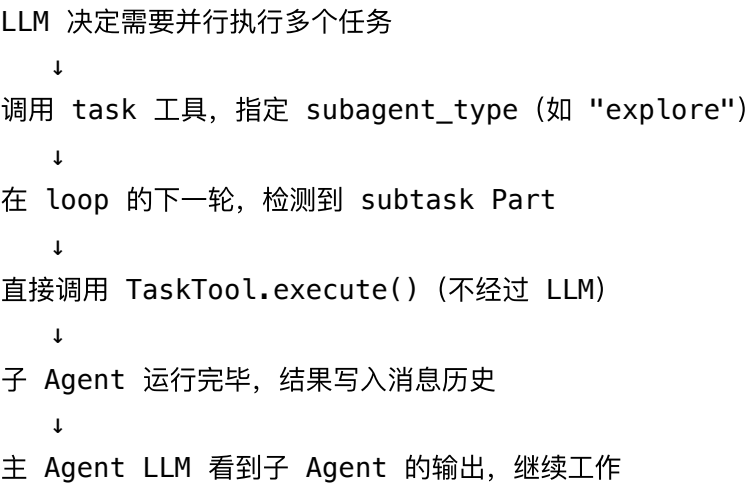
内置工具一览

工具 ID	文件	用途
bash	tool/bash.ts	执行 shell 命令
read	tool/read.ts	读取文件内容
write	tool/write.ts	创建/写入文件
edit	tool/edit.ts	编辑文件 (字符串替换)
apply_patch	tool/apply_patch.ts	应用 unified diff (GPT 专用)
glob	tool/glob.ts	文件名模式匹配
grep	tool/grep.ts	文本内容搜索
task	tool/task.ts	派发子 Agent 任务 ⭐
webfetch	tool/webfetch.ts	获取网页内容
websearch	tool/websearch.ts	网络搜索 (zen/EXA)
codesearch	tool/codesearch.ts	语义代码搜索
question	tool/question.ts	向用户提问
skill	tool/skill.ts	执行 skill 模板

工具 ID	文件	用途
todowrite	tool/todo.ts	写入任务列表
invalid	tool/invalid.ts	工具调用失败的 fallback
plan_enter/exit	tool/plan.ts	进入/退出 plan 模式（实验性）

TaskTool — 子 Agent 派发 ⭐

TaskTool 是实现多 Agent 协作的关键：



工具过滤逻辑

```
// apply_patch 和 edit/write 二选一
const usePatch = modelID.includes("gpt-") && !modelID.includes("oss")
if (t.id === "apply_patch") return usePatch
if (t.id === "edit" || t.id === "write") return !usePatch

// codesearch/websearch 仅 zen Provider 或 EXA flag
if (t.id === "codesearch" || t.id === "websearch") {
  return providerID === "opencode" || Flag.OPENCODE_ENABLE_EXA
}
```

工具执行的完整流程

```
resolveTools()
| 从 ToolRegistry 获取工具列表
| 将每个工具包装进 AI SDK tool(), 添加:
|   - 权限检查 (ctx.ask)
|   - 插件钩子 (tool.execute.before/after)
|   - MCP 工具
▼
LLM.stream()
| 传入 tools map
▼
工具被 LLM 调用
▼
tool.execute(args, ctx)
| ctx.ask() 检查权限 → 可能弹出权限询问
▼
返回 { output, title, metadata }
▼
processor 的 tool-result 事件写入数据库
```

8. 权限系统

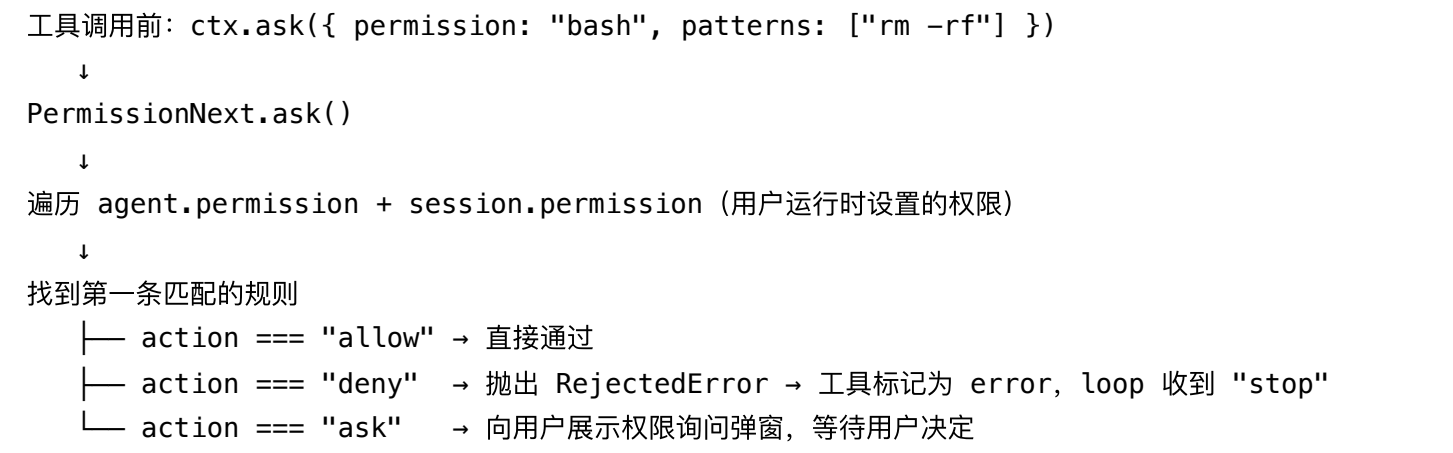
文件: `packages/opencode/src/permission/next.ts`

权限规则 (Ruleset)

每个 Agent 有一个 Ruleset (规则数组), 每条规则:

```
{
  permission: string, // 权限名 (工具名或特殊权限如 "doom_loop")
  action: "allow" | "deny" | "ask",
  pattern: string,    // glob 匹配 (如 "*.env", "*")
}
```

权限检查流程



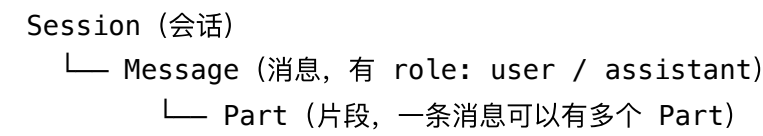
特殊权限名

权限名	触发时机
<code>doom_loop</code>	检测到死循环时
<code>external_directory</code>	访问项目目录外的文件
<code>question</code>	使用 <code>question</code> 工具
<code>plan_enter</code>	进入 <code>plan</code> 模式
<code>plan_exit</code>	退出 <code>plan</code> 模式

9. 消息与会话模型

文件: `packages/opencode/src/session/message-v2.ts`

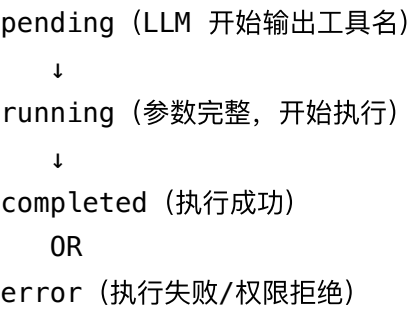
消息层级结构



Part 的类型

Part 类型	含义
text	文本内容
reasoning	思维链/推理过程
tool	工具调用（含状态: pending → running → completed/error）
file	附件（图片、代码文件等）
step-start	步骤开始标记（含文件快照 ID）
step-finish	步骤结束标记（含 token 消耗）
patch	文件变更 diff（本步骤修改了哪些文件）
compaction	上下文压缩标记
subtask	子 Agent 任务描述

ToolPart 的状态机



10. 上下文压缩（Compaction）

文件： packages/opencode/src/session/compaction.ts

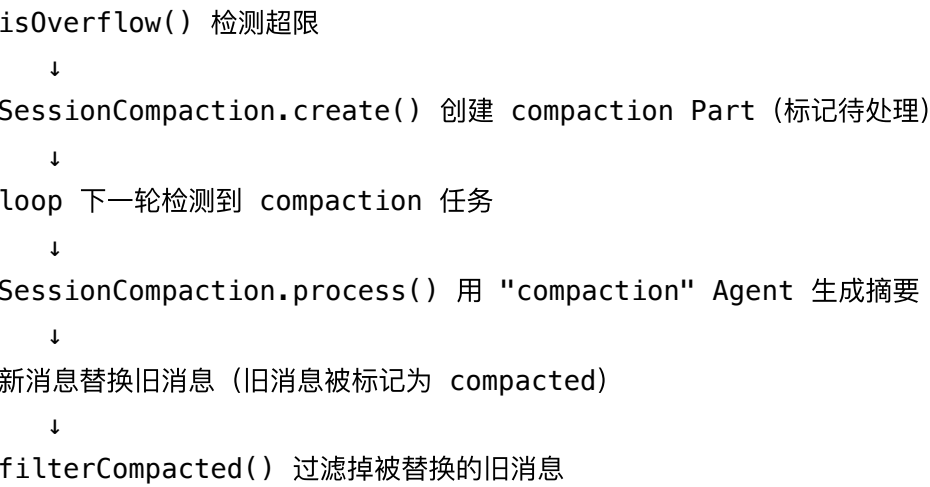
为什么需要压缩？

每个模型有 token 上限（如 Claude 200K）。长对话会超限。
压缩 = 用 AI 把旧消息总结成摘要，替换原始消息。

压缩触发时机

- 1. 自动触发： finish-step 事件后检查 token 用量，超过阈值 → 在 loop 下一轮执行
- 2. 手动触发： 用户点击"压缩上下文"

压缩流程



11. 插件系统（Plugin）

文件： packages/opencode/src/plugin/

插件通过"钩子（hook）"在各个关键节点注入逻辑：

主要钩子点

钩子名	触发时机
chat.params	LLM 调用前，可修改 temperature/topP 等参数
chat.headers	LLM 调用前，可添加自定义 HTTP headers
chat.message	用户消息创建后，可修改消息内容
tool.definition	工具初始化时，可修改工具描述和参数 schema
tool.execute.before	工具执行前
tool.execute.after	工具执行后，可修改输出

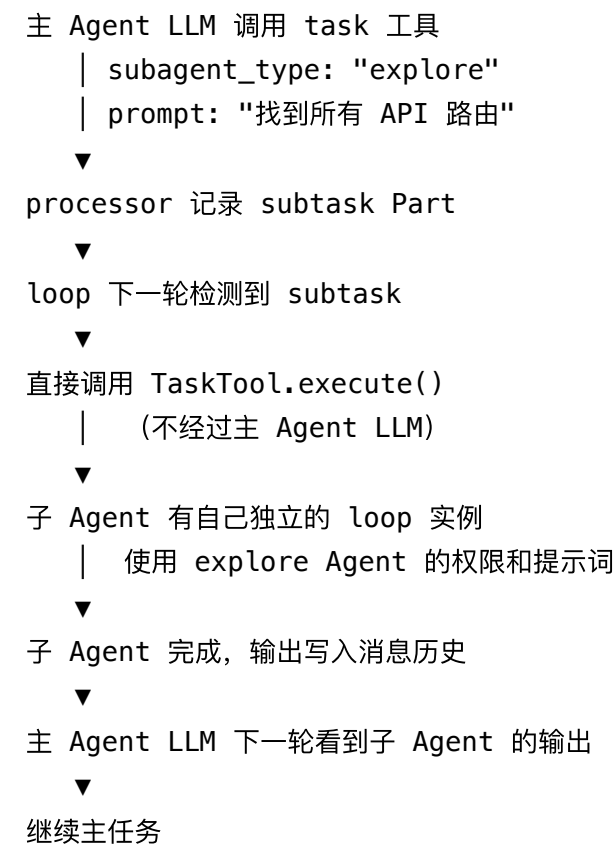
钩子名	触发时机
experimental.chat.system.transform	可修改系统提示词
experimental.text.complete	LLM 文本输出结束时，可修改最终文本
shell.env	shell 命令执行前，可注入环境变量
command.execute.before	/command 执行前

12. 关键数据流图

一次完整的用户请求流



子 Agent 调用流



13. 文件速查表

核心文件

文件	作用
session/prompt.ts	★ 主循环, 入口
session/processor.ts	★ LLM 流事件处理
session/llm.ts	LLM 调用封装
agent/agent.ts	Agent 定义与配置
tool/registry.ts	工具注册表
tool/task.ts	子 Agent 派发工具

文件	作用
session/message-v2.ts	消息/Part 数据模型
session/compaction.ts	上下文压缩
permission/next.ts	权限系统
provider/provider.ts	LLM Provider 管理

Session 相关

文件	作用
session/index.ts	Session CRUD, updateMessage/updatePart
session/system.ts	系统提示词（环境信息、Provider 提示词）
session/instruction.ts	指令提示词（从文件加载的自定义指令）
session/summary.ts	消息摘要生成
session/retry.ts	错误重试逻辑
session/status.ts	会话状态（idle/busy/retry）
session/revert.ts	会话回退

工具文件

文件	工具 ID
tool/bash.ts	bash
tool/read.ts	read
tool/write.ts	write
tool/edit.ts	edit
tool/glob.ts	glob
tool/grep.ts	grep
tool/task.ts	task

文件	工具 ID
tool/webfetch.ts	webfetch
tool/truncation.ts	输出截断（超长输出写文件）

基础设施

文件	作用
bus/index.ts	事件总线（内部发布/订阅）
id/id.ts	ULID 生成（时序递增 ID）
snapshot/index.ts	文件快照（用于生成 diff）
mcp/index.ts	MCP 服务器管理
lsp/index.ts	LSP 语言服务器集成
config/config.ts	配置文件读取
project/instance.ts	项目实例（单例状态管理）

推荐阅读顺序

1. **agent/agent.ts** — 先理解有哪些 Agent，权限怎么配置
2. **session/prompt.ts** 的 `loop()` 函数 — 整个系统的主干
3. **session/processor.ts** 的 `process()` 函数 — 理解流式处理
4. **session/llm.ts** 的 `stream()` 函数 — 理解如何调用 LLM
5. **tool/registry.ts** — 理解工具怎么被加载和过滤
6. **tool/task.ts** — 理解子 Agent 是怎么工作的
7. **session/compaction.ts** — 理解上下文压缩机制